

Programming in the large

↳ problem: managing complexity

Procedural programming ← ① handle exceptions gracefully
② test functions

Structure: (calculator of rational nr.)

Non-Ui functions (no input/print)

Functions to handle rational nr.

Functions to handle the calculator

stores
1) current value
2) history of prev. value

Only-Ui functions

```
def create_rational (numerator: int, denominator: int)
```

```
    pass # does nothing for now
```

or

```
    print (numerator, denominator)
```

or

```
    return [numerator, denominator]
```

```
g = create_rational (4, 0)
```

```
print(g)
```

⇒ [4, 0] not ok ⇒ message / don't create the list.

```
def create_rational (...)
```

```
    if denominator == 0:
```

```
        raise ValueError ("Cannot create ...")
```

```
    return [numerator, denominator]
```

```
g = None
```

try:

```
    g = create_rational (...)
```

except ValueError as ve:

except ValueError as e

print(re)

print(g)

getters/setters →

I want functions that don't print anything, but prepare the "canvas" for the operations. What will they return? → strings.

```
def get-numerator(g):  
    return g[0]
```

g:

0	1
num.	denom.

```
def get-denominator(g):  
    return g[1]
```

→ hint

```
def to-string(g) → str:  
    return str(get-numerator(g)) + "/" + str(get-denominator(g))
```

before we had just a list (ex: [3,4])

↓

now we have a string (ex: "3/4").

Test functions:

- call other functions in the program
- unit tests " (advanced)
- " should be independent
- should not return anything
- **assert**
- should not have parameters
- we want the error to be raised

↳ ex: test that nr. with denominator 0 ...

"first-class citizens" → can be used as variables, ...

call operator: ()

If we want 1 instead of 1/1 → string

#TODO

#FIXME

↳ `raise ValueError("denominator is 0")` - the line that actually raises exception

commans [option] (calculator) → the time that actually raises exception.

Rational calculator

- + add a rational nr. to the calculator
- u undo the last op.
- x exit

from math import gcd

```
def create_rational(num, denom):  
    if denom == 0:  
        raise ValueError("Fraction denominator 0")  
  
    d = gcd(num, denom)  
    num = num // d  
    denom = denom // d  
    return [num, denom]
```

} simplify fraction

```
def get_numerator(q):  
    return q[0]
```

```
def get_denominator(q):  
    return q[1]
```

$$\frac{3}{2} + \frac{2}{3} = \frac{3 \cdot 1 + 2 \cdot 1}{3 \cdot 2}$$

```
def add(q1, q2) → the sum of 2 rational nr.
```

```
    return create_rational(get_numerator(q1) * get_denominator(q2) +  
                           get_numerator(q2) * get_denominator(q1),  
                           get_denominator(q1) * get_denominator(q2))
```

```
def to_str(q)
```

```
    return str(get_numerator(q)) + "/" + str(get_denominator(q))
```

```
def create_calculator(): → set up the initial state of the calculator.
```

```
    return {'val': create_rational(0, 1), 'history': []} → dictionary
```

```
def get_total(calc): → return the current result
```

```
    return calc['val']
```

NON-VI FUNCTIONS
- functions to handle rational nr. -

FUNCTIONS TO HANDLE THE CALCULATOR

def set_total (calc, g): → set the current result

calc ['val'] = g

def add_calculator (calc, g): → add a nr to the current total

calc_total = get_total (calc)

history = calc ['history']

history.append (calc_total)

set_total (calc, add (calc_total, g))

def undo_calc (calc):

history = calc ['history']

if len (history) == 0:

return -1

calc ['val'] = history [-1] → ^{save} the most recent total from history

history.pop() → remove the most recent total

return 0

: [0, 1], [1, 2]

def print_menu():

print (" ... ")

print (" + ... ")

print (" u ... ")

print (" x ... ")

def add_calc_ui (calc):

m = input (" Numerator: ")

n = input (" Denominator: ")

add_calculator (calc, create_rational (int (m), int (n)))

def undo_calc_ui (calc):

success = undo_calc (calc)

if success < 0:

print ('Nothing to undo!')

def run():

commands = { '+': add_calc_ui, 'u': undo_calc_ui }

VI - FUNCTIONS

calculator = create_calculator() → make new calculator.

finish = False } keep looping until decide to stop.

while not finish: → show current total → to text

print('Total = ' + to_str(get_total(calculator)))

print_menu() → available commands

m = input().strip() → remove extra spaces from input

if m in commands.keys(): → if key in dictionary
 commands[m](calculator)

elif m == 'x':

 return

else:

 print("Invalid command")

run()